asio实现http服务器

Posted on 2023-06-23 19:54:29 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2RbaWyENb1b1trG4KoWHtScUHri)

简介

前文介绍了asio如何实现并发的长连接tcp服务器，今天介绍如何实现http服务器，在介绍实现http服务器之前，需要讲述下http报文头的格式，其实http报文头的格式就是为了避免我们之前提到的粘包现象，告诉服务器一个数据包的开始和结尾，并在包头里标识请求的类型如get或post等信息。

HTTP包头信息

一个标准的HTTP报文头通常由请求头和响应头两部分组成。

HTTP 请求头

HTTP请求头包括以下字段：

- **Request-line**：包含用于描述请求类型、要访问的资源以及所使用的HTTP版本的信息。
- **Host**：指定被请求资源的主机名或IP地址和端口号。
- **Accept**：指定客户端能够接收的媒体类型列表，用逗号分隔，例如 text/plain, text/html。
- **User-Agent**：客户端使用的浏览器类型和版本号，供服务器统计用户代理信息。
- **Cookie**：如果请求中包含cookie信息，则通过这个字段将cookie信息发送给Web服务器。
- **Connection**：表示是否需要持久连接（keep-alive）。

比如下面就是一个实际应用

```
1. GET /index.html HTTP/1.1
2. Host: www.example.com
3. Accept: text/html, application/xhtml+xml, */*
4. User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:123.0) Gecko/20100101 Firefox/123.0
5. Cookie: sessionid=abcdefg1234567
6. Connection: keep-alive
```

上述请求头包括了以下字段：

- **Request-line**：指定使用GET方法请求/index.html资源，并使用HTTP/1.1协议版本。
- **Host**：指定被请求资源所在主机名或IP地址和端口号。
- **Accept**：客户端期望接收的媒体类型列表，本例中指定了text/html、application/xhtml+xml和任意类型的文件（/）。
- **User-Agent**：客户端浏览器类型和版本号。
- **Cookie**：客户端发送给服务器的cookie信息。
- **Connection**：客户端请求后是否需要保持长连接。

HTTP 响应头

HTTP响应头包括以下字段：

- **Status-line**：包含协议版本、状态码和状态消息。
- **Content-Type**：响应体的MIME类型。
- **Content-Length**：响应体的字节数。
- **Set-Cookie**：服务器向客户端发送cookie信息时使用该字段。
- **Server**：服务器类型和版本号。
- **Connection**：表示是否需要保持长连接（keep-alive）。

在实际的HTTP报文头中，还可以包含其他可选字段。
如下是一个http响应头的示例

```
1. HTTP/1.1 200 OK
2. Content-Type: text/html; charset=UTF-8
3. Content-Length: 1024
4. Set-Cookie: sessionid=abcdefg1234567; HttpOnly; Path=/
5. Server: Apache/2.2.32 (Unix) mod_ssl/2.2.32 OpenSSL/1.0.1e-fips mod_bwlimited/1.4
6. Connection: keep-alive
```

上述响应头包括了以下字段：

- **Status-line**：指定HTTP协议版本、状态码和状态消息。
- **Content-Type**：指定响应体的MIME类型及字符编码格式。
- **Content-Length**：指定响应体的字节数。
- **Set-Cookie**：服务器向客户端发送cookie信息时使用该字段。
- **Server**：服务器类型和版本号。
- **Connection**：服务器是否需要保持长连接。

客户端的编写

客户端每次发送数据都要携带头部信息，所以为了减少每次重新构造头部的开销，我们在客户端的构造函数里将头部信息构造好，作为一个成员放入客户端的类成员里。

```
client(boost::asio::io_context& io_context,
    const std::string& server, const std::string& path)
    : resolver_(io_context),
    socket_(io_context)
{
    // Form the request. We specify the "Connection: close" header so that the
    // server will close the socket after transmitting the response. This will
    // allow us to treat all data up until the EOF as the content.
    std::ostream request_stream(&request_);
    request_stream << "GET " << path << " HTTP/1.0\r\n";
    request_stream << "Host: " << server << "\r\n";
    request_stream << "Accept: */*\r\n";
    request_stream << "Connection: close\r\n\r\n";

    size_t pos = server.find(":");
    std::string ip = server.substr(0, pos);
    std::string port = server.substr(pos + 1);

    // Start an asynchronous resolve to translate the server and service names
    // into a list of endpoints.
    resolver_.async_resolve(ip, port,
        boost::bind(&client::handle_resolve, this,
            boost::asio::placeholders::error,
            boost::asio::placeholders::results));
}
```

我们的客户端构造了一个 `request_` 成员变量，依次写入请求的路径，主机地址，期望接受的媒体类型，以及每次收到请求后断开连接，也就是短链接的方式。

接着又异步解析ip和端口，解析成功后调用 `handle_resolve` 函数。
`handle_resolve` 函数里异步处理连接

```
void handle_resolve(const boost::system::error_code& err,
    const tcp::resolver::results_type& endpoints)
{
    if (!err)
    {
        // Attempt a connection to each endpoint in the list until we
        // successfully establish a connection.
        boost::asio::async_connect(socket_, endpoints,
            boost::bind(&client::handle_connect, this,
                boost::asio::placeholders::error));
    }
    else
    {
        std::cout << "Error: " << err.message() << "\n";
    }
}
```

处理连接

```
void handle_connect(const boost::system::error_code& err)
{
    if (!err)
    {
        // The connection was successful. Send the request.
        boost::asio::async_write(socket_, request_,
            boost::bind(&client::handle_write_request, this,
                boost::asio::placeholders::error));
    }
    else
    {
        std::cout << "Error: " << err.message() << "\n";
    }
}
```

在连接成功后，我们首先将头部信息发送给服务器,发送完成后监听对端发送的数据

```
1.  void handle_write_request(const boost::system::error_code& err)
2.  {
3.      if (!err)
4.      {
5.          // Read the response status line. The response_ streambuf will
6.          // automatically grow to accommodate the entire line. The growth may be
7.          // limited by passing a maximum size to the streambuf constructor.
8.          boost::asio::async_read_until(socket_, response_, "\r\n",
9.              boost::bind(&client::handle_read_status_line, this,
10.                 boost::asio::placeholders::error));
11.     }
12.     else
13.     {
14.         std::cout << "Error: " << err.message() << "\n";
15.     }
16. }
```

当收到对方数据时，先解析响应的头部信息

```
void handle_read_status_line(const boost::system::error_code& err)
{
    if (!err)
    {
        // Check that response is OK.
        std::istream response_stream(&response_);
        std::string http_version;
        response_stream >> http_version;
        unsigned int status_code;
        response_stream >> status_code;
        std::string status_message;
        std::getline(response_stream, status_message);
        if (!response_stream || http_version.substr(0, 5) != "HTTP/")
        {
            std::cout << "Invalid response\n";
            return;
        }
        if (status_code != 200)
        {
            std::cout << "Response returned with status code ";
            std::cout << status_code << "\n";
            return;
        }

        // Read the response headers, which are terminated by a blank line.
        boost::asio::async_read_until(socket_, response_, "\r\n\r\n",
            boost::bind(&client::handle_read_headers, this,
                boost::asio::placeholders::error));
    }
    else
    {
        std::cout << "Error: " << err << "\n";
    }
}
```

上面的代码先读出HTTP版本，以及返回的状态码，如果状态码不是200，则返回，是200说明响应成功。接下来把所有的头部信息都读出来。

```
1.  void handle_read_headers(const boost::system::error_code& err)
2.  {
3.     if (!err)
4.     {
5.        // Process the response headers.
6.        std::istream response_stream(&response_);
7.        std::string header;
8.        while (std::getline(response_stream, header) && header != "\r")
9.           std::cout << header << "\n";
10.       std::cout << "\n";
11.
12.       // Write whatever content we already have to output.
13.       if (response_.size() > 0)
14.          std::cout << &response_;
15.
16.       // Start reading remaining data until EOF.
17.       boost::asio::async_read(socket_, response_,
18.          boost::asio::transfer_at_least(1),
19.          boost::bind(&client::handle_read_content, this,
20.             boost::asio::placeholders::error));
21.    }
22.    else
23.    {
24.       std::cout << "Error: " << err << "\n";
25.    }
26. }
```

上面的代码逐行读出头部信息，然后读出响应的内容，继续监听读事件读取相应的内容，直到接收到EOF信息，也就是对方关闭，继续监听读事件是因为有可能是长连接的方式，当然如果是短链接，则服务器关闭连接后，客户端也是通过异步函数读取EOF进而结束请求。

```
void handle_read_content(const boost::system::error_code& err)
{
    if (!err)
    {
        // Write all of the data that has been read so far.
        std::cout << &response_;

        // Continue reading remaining data until EOF.
        boost::asio::async_read(socket_, response_,
            boost::asio::transfer_at_least(1),
            boost::bind(&client::handle_read_content, this,
                boost::asio::placeholders::error));
    }
    else if (err != boost::asio::error::eof)
    {
        std::cout << "Error: " << err << "\n";
    }
}
```

在主函数中调用客户端请求服务器信息, 请求的路由地址为 `/`

```
int main(int argc, char* argv[])
{
    try
    {
        boost::asio::io_context io_context;
        client c(io_context, "127.0.0.1:8080", "/");
        io_context.run();
        getchar();
    }
    catch (std::exception& e)
    {
        std::cout << "Exception: " << e.what() << "\n";
    }

    return 0;
}
```

服务器设计

为了方便理解，我们从服务器的调用流程讲起

```
1. int main(int argc, char* argv[])
2. {
3.     try
4.     {
5.         std::filesystem::path path = std::filesystem::current_path() / "res";
6.
7.         // 使用 std::cout 输出拼接后的路径
8.         std::cout << "Path: " << path.string() << '\n';
9.         std::cout << "Usage: http_server <127.0.0.1> <8080> "<< path.string() <<"\n";
10.        // Initialise the server.
11.        http::server::server s("127.0.0.1", "8080", path.string());
12.
13.        // Run the server until stopped.
14.        s.run();
15.    }
16.    catch (std::exception& e)
17.    {
18.        std::cerr << "exception: " << e.what() << "\n";
19.    }
20.
21.    return 0;
22. }
```

主函数里构造了一个server对象，然后调用了run函数使其跑起来。
run函数其实就是调用了server类成员的ioservice

```
1. void server::run()
2. {
3.     io_service_.run();
4. }
```

server类的构造函数里初始化一些成员变量，比如acceptor连接器，绑定了终止信号，并且监听对端连接

```
server::server(const std::string& address, const std::string& port,
    const std::string& doc_root)
        : io_service_(),
        signals_(io_service_),
        acceptor_(io_service_),
        connection_manager_(),
        socket_(io_service_),
        request_handler_(doc_root)
    {
        signals_.add(SIGINT);
        signals_.add(SIGTERM);
#if defined(SIGQUIT)
        signals_.add(SIGQUIT);
#endif
        do_await_stop();
        boost::asio::ip::tcp::resolver resolver(io_service_);
        boost::asio::ip::tcp::endpoint endpoint = *resolver.resolve({ address, port });
        acceptor_.open(endpoint.protocol());
        acceptor_.set_option(boost::asio::ip::tcp::acceptor::reuse_address(true));
        acceptor_.bind(endpoint);
        acceptor_.listen();
        do_accept();
    }
```

接收连接

```
void server::do_accept()
{
    acceptor_.async_accept(socket_,
        [this](boost::system::error_code ec)
        {

            if (!acceptor_.is_open())
            {
                return;
            }

            if (!ec)
            {
                connection_manager_.start(std::make_shared<connection>(
                        std::move(socket_), connection_manager_, request_handler_));
                  }

                do_accept();
            });
        }
```

接收函数里通过 `connection_manager_` 启动了一个新的连接，用来处理读写函数。
处理方式和我们之前的写法类似，只是我们之前管理连接用的server，这次用的conneciton_manager

```
void connection_manager::start(connection_ptr c)
{
    connections_.insert(c);
    c->start();
}
```

start函数里处理读写

```
void connection::start()
{
    do_read();
}
```

处理读数据比较复杂，我们分部分解释

```
1. void connection::do_read()
2. {
3.     auto self(shared_from_this());
4.     socket_.async_read_some(boost::asio::buffer(buffer_),
5.         [this, self](boost::system::error_code ec, std::size_t bytes_transferred)
6.     {
7.         if (!ec)
8.         {
9.             request_parser::result_type result;
10.            std::tie(result, std::ignore) = request_parser_.parse(
11.                request_, buffer_.data(), buffer_.data() + bytes_transferred);
12.
13.            if (result == request_parser::good)
14.            {
15.                request_handler_.handle_request(request_, reply_);
16.                do_write();
17.            }
18.            else if (result == request_parser::bad)
19.            {
20.                reply_ = reply::stock_reply(reply::bad_request);
21.                do_write();
22.            }
23.            else
24.            {
25.                do_read();
26.            }
27.        }
28.        else if (ec != boost::asio::error::operation_aborted)
29.        {
30.            connection_manager_.stop(shared_from_this());
31.        }
32.    });
33. }
```

通过 `request_parser_` 解析请求，然后根据请求结果选择处理请求还是返回错误。

```
std::tuple<result_type, InputIterator> parse(request& req,
                InputIterator begin, InputIterator end)
{
    while (begin != end)
    {
        result_type result = consume(req, *begin++);
        if (result == good || result == bad)
            return std::make_tuple(result, begin);
    }
    return std::make_tuple(indeterminate, begin);
}
```

parse是解析请求的函数，内部调用了consume不断处理请求头中的数据，其实就是一个逐行解析的过程, consume函数很长，这里就不解释了，其实就是每解析一行就更改一下状态，这样可以继续解析。具体可以看看源码。

在consume()函数中，根据每个字符输入的不同情况，判断当前所处状态state_，进而执行相应的操作，包括：

- 将HTTP请求方法、URI和HTTP版本号解析到request结构体中。
- 解析每个请求头部字段的名称和值，并将其添加到request结构体中的headers vector中。
- 如果输入字符为 `\r\n` ，则修改状态以开始下一行的解析。

最后，返回一个枚举类型request_parser::result_type作为解析结果，包括indeterminate、good和bad三种状态。其中，indeterminate表示还需要继续等待更多字符输入；good表示成功解析出了一个完整的HTTP请求头部；bad表示遇到无效字符或格式错误，解析失败。

解析完成头部后会调用处理请求的函数,这里只是简单的写了一个作为资源服务器解析资源请求的逻辑，具体可以看源码。

```
1. void request_handler::handle_request(const request& req, reply& rep)
2. {
3.   // Decode url to path.
4.   std::string request_path;
5.   if (!url_decode(req.uri, request_path))
6.   {
7.     rep = reply::stock_reply(reply::bad_request);
8.     return;
9.   }
10.  // Request path must be absolute and not contain "..".
11.  if (request_path.empty() || request_path[0] != '/'
12.    || request_path.find("..") != std::string::npos)
13.  {
14.    rep = reply::stock_reply(reply::bad_request);
15.    return;
16.  }
17.
18.  // If path ends in slash (i.e. is a directory) then add "index.html".
19.  if (request_path[request_path.size() - 1] == '/')
20.  {
21.    request_path += "index.html";
22.  }
23.
24.  // Determine the file extension.
25.  std::size_t last_slash_pos = request_path.find_last_of("/");
26.  std::size_t last_dot_pos = request_path.find_last_of(".");
27.  std::string extension;
28.  if (last_dot_pos != std::string::npos && last_dot_pos > last_slash_pos)
29.  {
30.    extension = request_path.substr(last_dot_pos + 1);
31.  }
32.
33.  // Open the file to send back.
34.  std::string full_path = doc_root_ + request_path;
35.  std::ifstream is(full_path.c_str(), std::ios::in | std::ios::binary);
36.  if (!is)
37.  {
38.    rep = reply::stock_reply(reply::not_found);
39.    return;
40.  }
41.
42.  // Fill out the reply to be sent to the client.
```

```
43.    rep.status = reply::ok;
44.    char buf[512];
45.    while (is.read(buf, sizeof(buf)).gcount() > 0)
46.      rep.content.append(buf, is.gcount());
47.      rep.headers.resize(2);
48.      rep.headers[0].name = "Content-Length";
49.      rep.headers[0].value = std::to_string(rep.content.size());
50.      rep.headers[1].name = "Content-Type";
51.      rep.headers[1].value = mime_types::extension_to_type(extension);
52.    }
```

上述代码根据url中的 `.` 来做切割，获取请求的文件类型，然后根据 `/` 切割url，获取资源目录，最后返回资源文件。
如果你想实现普通的路由请求返回json或者text格式，可以重写处理请求的逻辑。

总结

本文介绍了如何使用asio实现http服务器，具体可以查看下方源码，其实这些仅作为了解即可，不推荐从头造轮子，我们可以用一些C++ 成熟的http服务库比如beast，下一节再介绍。

视频连接https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101)

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(2971) (JavaScript:void(0);) 评论(1) (JavaScript:void(0);) 喜欢(535) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论



dong

2023-09-06 10:20:24

[IO复用,性能提升在哪里]

服务器接收流程:
1.绑定端口等待建立socket连接
2.accept建立对话socket
3.对话socket,循环async_read_some,读取数据包,解析html数据包 [线程池+协程]
4.对话socket,循环async_write_some,生成html数据包,发送数据包 [线程池+协程]
5.结束对话,关闭socket

疑问:
IO复用,是优化在哪里,流程个人理解也是一样,比如有新对话socket,它是通过回调处理,然后也是在回调里面循环读取数据包,再发送.

♡ 喜欢(0) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看 ⩔

没有更多评论了,点击收起 ⩓ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态
gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数
李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)